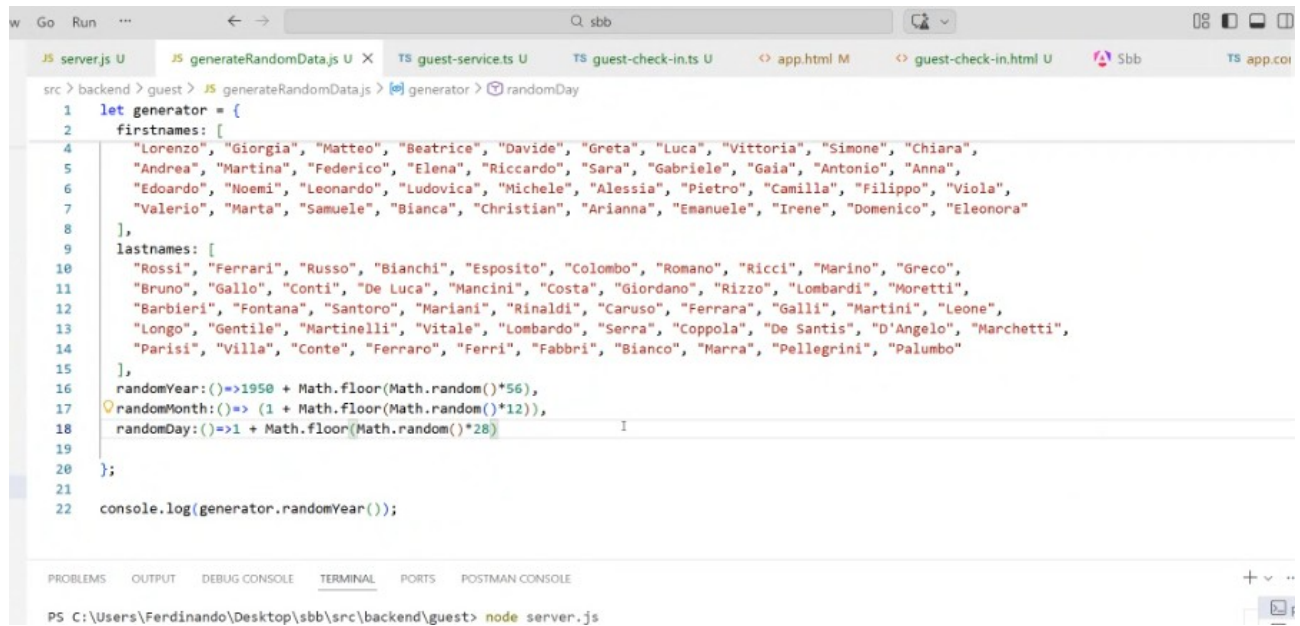


I dati di mock sono fondamentali per qualunque progetto reale.

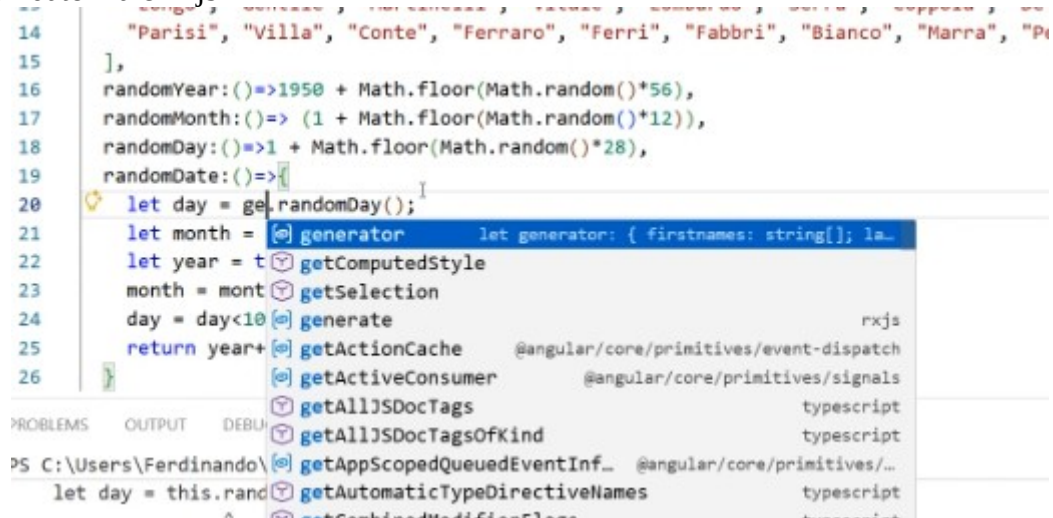
Generatori di dati casuali



```
1 let generator = {
2   firstnames: [
3     "Lorenzo", "Giorgia", "Matteo", "Beatrice", "Davide", "Greta", "Luca", "Vittoria", "Simone", "Chiara",
4     "Andrea", "Martina", "Federico", "Elena", "Riccardo", "Sara", "Gabriele", "Gaia", "Antonio", "Anna",
5     "Edoardo", "Noemi", "Leonardo", "Ludovica", "Michele", "Alessia", "Pietro", "Camilla", "Filippo", "Viola",
6     "Valerio", "Marta", "Samuele", "Bianca", "Christian", "Arianna", "Emanuele", "Irene", "Domenico", "Eleonora"
7   ],
8   lastnames: [
9     "Rossi", "Ferrari", "Russo", "Bianchi", "Esposito", "Colombo", "Romano", "Ricci", "Marino", "Greco",
10    "Bruno", "Gallo", "Conti", "De Luca", "Mancini", "Costa", "Giordano", "Rizzo", "Lombardi", "Moretti",
11    "Barbieri", "Fontana", "Santoro", "Mariani", "Rinaldi", "Caruso", "Ferrara", "Galli", "Martini", "Leone",
12    "Longo", "Gentile", "Martinelli", "Vitale", "Lombardo", "Serra", "Coppola", "De Santis", "D'Angelo", "Marchetti",
13    "Parisi", "Villa", "Conte", "Ferraro", "Ferri", "Fabbri", "Bianco", "Marra", "Pellegrini", "Palumbo"
14  ],
15  randomYear: () => 1950 + Math.floor(Math.random()*56),
16  randomMonth: () => (1 + Math.floor(Math.random()*12)),
17  randomDay: () => 1 + Math.floor(Math.random()*28),
18  randomDate: () => {
19  },
20 };
21 console.log(generator.randomYear());
```

PS C:\Users\Ferdinando\Desktop\sbb\src\backend\guest> node server.js

il this è bindato male in js



```
14 "Parisi", "Villa", "Conte", "Ferraro", "Ferri", "Fabbri", "Bianco", "Marra", "Pellegrini", "Palumbo"
15 ],
16 randomYear: () => 1950 + Math.floor(Math.random()*56),
17 randomMonth: () => (1 + Math.floor(Math.random()*12)),
18 randomDay: () => 1 + Math.floor(Math.random()*28),
19 randomDate: () => {
20 },
21 let day = generator.randomDay();
22 let month = generator.randomMonth();
23 let year = generator.randomYear();
24 month = month < 10 ? "0" + month : month;
25 day = day < 10 ? "0" + day : day;
26 return year + "-" + month + "-" + day;
```

let generator: { firstnames: string[]; lastnames: string[]; randomYear: () => number; randomMonth: () => number; randomDay: () => number; randomDate: () => string; };

```
5   "Andrea", "Martina", "Federico", "Elena", "Riccardo", "Sara", "Gabriele", "Gaia", "Antonio", "Anna",
6   "Edoardo", "Noemi", "Leonardo", "Ludovica", "Michele", "Alessia", "Pietro", "Camilla", "Filippo", "Viola",
7   "Valerio", "Marta", "Samuele", "Bianca", "Christian", "Arianna", "Emanuele", "Irene", "Domenico", "Eleonora"
8 ],
9   lastnames: [
10    "Rossi", "Ferrari", "Russo", "Bianchi", "Esposito", "Colombo", "Romano", "Ricci", "Marino", "Greco",
11    "Bruno", "Gallo", "Conti", "De Luca", "Mancini", "Costa", "Giordano", "Rizzo", "Lombardi", "Moretti",
12    "Barbieri", "Fontana", "Santoro", "Mariani", "Rinaldi", "Caruso", "Ferrara", "Galli", "Martini", "Leone",
13    "Longo", "Gentile", "Martinelli", "Vitale", "Lombardo", "Serra", "Coppola", "De Santis", "D'Angelo", "Marchetti",
14    "Parisi", "Villa", "Conte", "Ferraro", "Ferri", "Fabbri", "Bianco", "Marra", "Pellegrini", "Palumbo"
15  ],
16  randomYear: () => 1950 + Math.floor(Math.random()*56),
17  randomMonth: () => (1 + Math.floor(Math.random()*12)),
18  randomDay: () => 1 + Math.floor(Math.random()*28),
19  randomDate: () => {
20    let day = generator.randomDay();
21    let month = generator.randomMonth();
22    let year = generator.randomYear();
23    month = month < 10 ? "0"+month : month;
24    day = day < 10 ? "0"+day : day;
25    return year+"-"+month+"-"+day;
26  }
27 };
```

Sgancia questo v

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

PS C:\Users\Ferdinando\Desktop\sbb\src\backend\guest> node generateRandomData.js

at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
at node:internal/main/run_main_module:33:47

Node.js v24.13.1

PS C:\Users\Ferdinando\Desktop\sbb\src\backend\guest> node generateRandomData.js

1960-10-01

Il file `generateRandomData.js` costruisce un oggetto `generator` che genera dati anagrafici casuali. Contiene due array con nomi e cognomi italiani, e quattro funzioni: `randomYear` produce un anno tra 1950 e 2006, `randomMonth` un mese tra 1 e 12, `randomDay` un giorno tra 1 e 28, e `randomDate` le combina restituendo una stringa in formato `YYYY-MM-DD` con zero padding sui valori a singola cifra.

È stata aggiunta la funzione `randomElement`, che riceve un array come parametro e restituisce un elemento casuale al suo interno.

Funziona generando un indice casuale tra 0 e la lunghezza dell'array meno 1, usando la stessa tecnica con `Math.floor` e `Math.random()` già vista nelle funzioni precedenti. Moltiplicare per `array.length` garantisce che l'indice generato sia sempre valido, qualunque sia la dimensione dell'array passato. Poi semplicemente restituisce l'elemento in quella posizione.

Questa funzione è quella che verrà usata per pescare un nome casuale da `firstnames` e un cognome casuale da `lastnames`, rendendo il generatore finalmente completo e riutilizzabile.

```
randomElement: (array) => {
  let randomPos = Math.floor(Math.random()*array.length); // numero casuale fra 0 e length dell'array - 1
  return array[randomPos]
},
randomDate: () => {
```

```

    },
    randomPerson:(id)=>
    {
        return {
            "id":id,
            "firstName":generator.randomElement(generator.firstnames),
            "lastName":generator.randomElement(generator.lastnames),
            "dob":generator.randomDate()
        }
    }
};

for(let i=1;i<1000;i++)
  console.log(generator.randomPerson(i));

```

Questa è la funzione finale che mette tutto insieme: `randomPerson` riceve un `id` come parametro e restituisce un oggetto che rappresenta una persona fittizia completa.

L'oggetto costruito ha quattro campi: `id` è semplicemente il numero passato come argomento, `firstName` è un nome pescato casualmente dall'array `firstnames` tramite `randomElement`, `lastName` un cognome pescato allo stesso modo da `lastnames`, e `dob` (date of birth) è una data di nascita generata da `randomDate`.

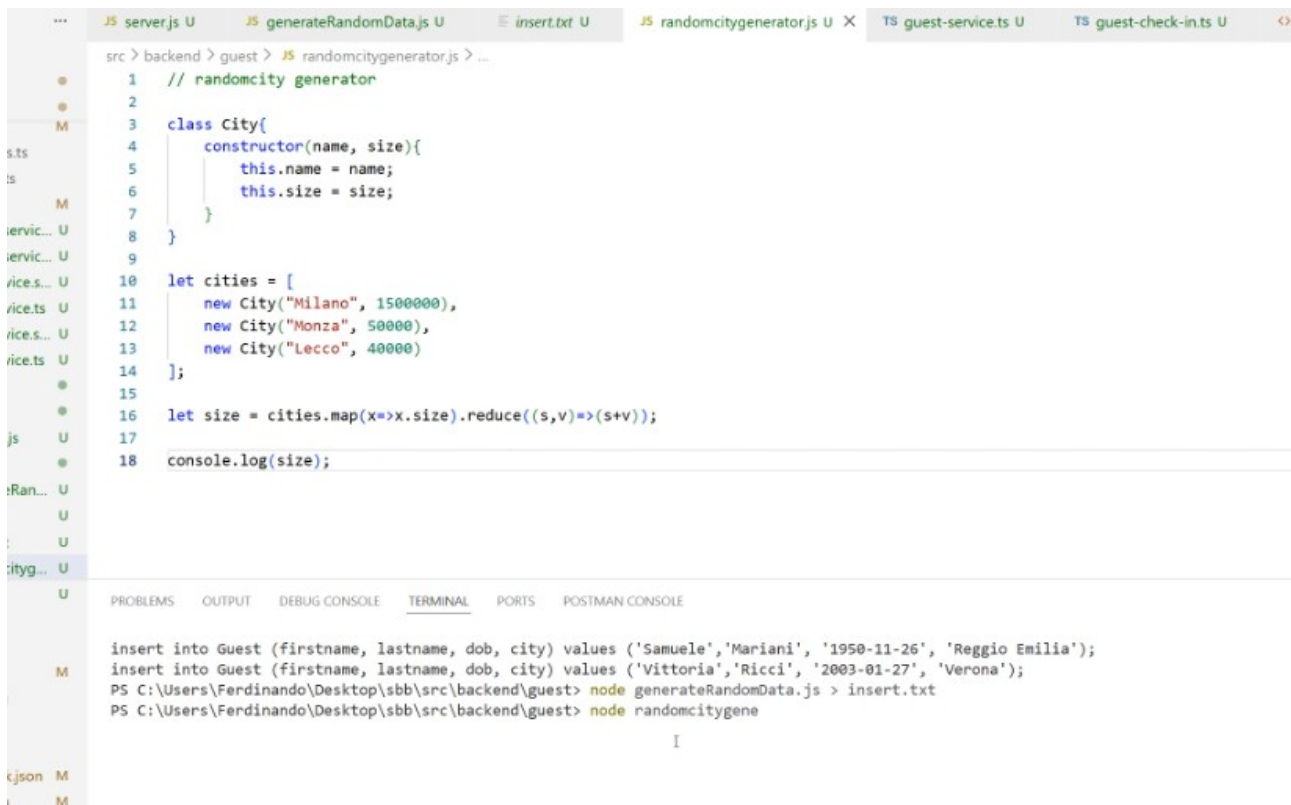
In fondo al file c'è un ciclo `for` che va da 1 a 999 e chiama `randomPerson(i)` mille volte, stampando in console mille persone fittizie diverse. Questo è essenzialmente il punto di arrivo dell'intero file: generare in massa dati anagrafici realistici da usare per popolare un database o testare un'applicazione.

//fare in modo tale che la probabilità di avere una città sia proporzionale ad uno dei suoi abitanti.

I dati generati sono uniformi e non mi sta bene, per logica milano dovrebbe avere 10.000 le persone di villasanta.

Scaricare online la lista di tutte le città italiane con popolazione.

Somma di frequenze:



```
src > backend > guest > JS randomcitygenerator.js > ...
1 // randomcity generator
2
3 class City{
4   constructor(name, size){
5     this.name = name;
6     this.size = size;
7   }
8 }
9
10 let cities = [
11   new City("Milano", 1500000),
12   new City("Monza", 50000),
13   new City("Lecco", 40000)
14 ];
15
16 let size = cities.map(x=>x.size).reduce((s,v)=>(s+v));
17
18 console.log(size);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
insert into Guest (firstname, lastname, dob, city) values ('Samuele','Mariani', '1950-11-26', 'Reggio Emilia');
insert into Guest (firstname, lastname, dob, city) values ('Vittoria','Ricci', '2003-01-27', 'Verona');
PS C:\Users\Ferdinando\Desktop\sbb\src\backend\guest> node generateRandomData.js > insert.txt
PS C:\Users\Ferdinando\Desktop\sbb\src\backend\guest> node randomcitygene
```

calcolare la performance di questo algoritmo, questa maniera è sequenziale.

```
26
27
28 // http://localhost:3001/api/cities/random
29
30 // http://localhost:3001/api/cities/byname/villasanta
```

Programma che genera una città casuale in base ai suoi cittadini da usare in altri programmi.